

PalmSource®

Conference and Expo



Targeting All Palm Powered™ Devices

Michael Knopp
Development Support Engineer
Palm, Inc.

Agenda

- Learn how to take advantage of licensee-specific hardware
- Specifics of coding for licensee hardware
- Obtaining and using licensee SDKs
- Extensible coding for detection and usage of licensee technology



Scope of this session

- Targeting or expanding a Palm OS® solution to utilize licensee technology
- Each licensee utilizes unique technology and targets specific markets, we will discuss technology
- Licensee agnostic: developers can decide which technologies best fit their respective solutions
- Applications should work on all licensee devices
  - Use features if available but do not require them
  - If required, alert the user of potential issues
  - Adhere to the APIs and do not make assumptions

Targeting specific licensee technology

- **HandEra**
  - Virtual Graffiti, 240 x 320, Jog Wheel, Secure Digital / MMC slot, CompactFlash™ slot
- **Handspring**
  - Springboard modules
- **Kyocera**
  - Signaling, Telephony, Phone module APIs
- **Palm**
  - Secure Digital / MultiMediaCard slot, SDIO, Universal Connector
- **Sony**
  - 320x320, Jog Dial, Memory Stick
- **Symbol**
  - Scanner APIs, LAN APIs, Printing APIs, Magnetic Strip Reader APIs

Additional Palm Powered devices

- **Acer s10**
  - Chinese-native Palm OS®
  - audio recording
  - MP3 playback
  - dual Memory Stick



- **Samsung I300 Palm Powered phone**
  - Voice Dialing
  - 240 x 160 pixels
  - virtual Graffiti area



Common across all licensee devices

- Palm OS:
  - Target the OS
    - require a minimum OS if appropriate and alert users if they do not meet the requirement, fail gracefully.
- Features and Hardware:
  - Never assume presence of a feature
  - Initialize the app by polling features and libraries if necessary
  - Never assume specifics of hardware
  - Licensees provide documentation and APIs
  - Things will change (really, they will!)

Discussion of Devices and APIs

- We will not be discussing the entire scope of each API
- We will touch on as many APIs as possible
  - Detection
    - How can I determine if a specific feature is present ?
  - Initialization
    - How can I open/query a specific feature ?
  - Usage
    - Now that I have the feature, what can I do ?
- If a specific feature is of interest, proceed to obtain the appropriate SDK or documentation for detailed information.

Introduction to the Feature Manager

- Many of the detection samples we will discuss utilize the Feature Manager.
- A feature is a 32-bit value that has specific meaning to both the feature publisher and to users of that feature.
- Each feature is identified by a feature creator and a feature number:
  - creator: unique creator ID, typically use application creator ID
  - Number: any 16-bit value to distinguish between different features of a particular creator
- A published feature remains present until it is explicitly unregistered or the device is reset.
- A feature published by an application is maintained even after the application quits.

```
// Detecting Palm OS version: See if we're on ROM version 3.1 or later.
FtrGet(sysFtrCreator, sysFtrNumROMVersion, &romVersion);
if (romVersion >= sysMakeROMVersion(3, 1, 0, sysROMStageRelease, 0)) {
....
}
```

Handera

http://www.handera.com/support/developsupport.asp



- **HandEra 330**
  - Palm OS® 3.5.3 with extensions for CompactFlash™,Secure Digital, MMC
  - Features Include: 240x320, Virtual Graffiti™, two expansion slots
- **TRGpro**
  - Palm OS 3.5.1
  - Features Include: CompactFlash™ Expansion Slot, Enhanced Audio
- **Development Tools include**
  - HandEra 330 HDK
  - HandEra 330 SDK
  - TRGpro SDK (included in Handera SDK): CF slot (VFS calls), Audio playback
  - ROMs are available for Handera 330 release and TRGpro release
  - Handera 330 SDK 1.05 is included in CodeWarrior for Palm OS v 8.0
- **Sample Code includes (HandEra 330 SDK)**
  - high-res icons, bitmaps and forms
  - resizing and rotating the display
  - one hand operation
  - voice recording
  - Silkscreen modification
  - icon scaling



Handera APIs

- **Quarter VGA (QVGA) 240 x 320**
  - Ensuring the VGA Extension is present

```
if FtrGet(TRGSysFtrID, TRGVgaFtrNum, &version) == 0)
{ // Extension is present }
```

  - Setting the application screen mode to either Scale To Fit or 1-1 Mapping

```
VgaSetScreenMode(screenMode1To1, rotateModeNone);   // before FrmGotoForm
FrmGotoForm(NewForm);
```

- **Jog Wheel and Auxiliary buttons**
  - Handled the same as any other keyDownEvent and defined in *TrgChars.h*

Action	***Key***
Jog Wheel rotated up	vchrPrevField
Jog Wheel rotated down	vchrNextField
Jog Wheel pressed	chrCarriageReturn
Auxiliary button pressed	chrEscape

- **MMC/SD and CF slots**
  - Handera 330 implements both the Palm OS 4.0 VFS and Expansion APIs
  - Behavior of certain VFS/Expansion functions is altered (documented in Programming Companion)

Handspring

http://www.handspring.com/developers/index.jhtml



- **Visor Line**
  - Prism (OS 3.5.2H1), Edge (OS 3.5.2.H3), Pro (OS 3.5.2.H3), Neo (OS 3.5.2.H3)
  - Features Include: Springboard expansion
- **Treo Line**
  - Treo 180 (Palm OS 3.5.2H)
  - Features Include: Jog rocker, SMS messaging, Wireless Web, mobile Phone
- **Development Tools include**
  - Handspring GNU Tools
  - POSE 3.1H3 supports all Visors and contains initial support for the Treo
  - ROMs available via ROM-transfer or through faxing a License
  - Treo Development technical documentation and development guide
  - Handspring DK 1.0 is included in CodeWarrior for Palm OS v 8.0
- **Sample Code includes (developer web page)**
  - baseline project for Springboard module application
  - support of 16-bit color in an application
  - localizing applications using the Handspring overlay utility
  - SMS send, receive, and message display
  - Checking for and using voice dialing services



Handspring APIs

- **Checking for Presence and Version of Handspring extensions**
  - constants defined in *HsExt.h*

```
err = FtrGet (hsFtrCreator, hsFtrIDVersion, &value);
if (!err)
{
   // We can now safely make Handspring specific API calls
}
```

Kyocera

http://www.kyocera-wireless.com/partner/partner.htm





- **SmartPhone Series**
  - Kyocera QCP 6035 smartphone: Palm OS 3.5
  - Features Include: phone keypad flip, CDMA interface, IS-99 (Async/Fax)
- **Development Tools include**
  - Kyocera SmartPhone SDK: API manual, CodeWarrior project files
  - Header files for signal, telephony, and registry APIs
- **Sample Code includes (SmartPhone SDK)**
  - SampleDialer: simple phone dialer
  - MakeCall: allows users to make voice calls
  - MinuteCounter: monitors talk time/month and warns when limit is reached
  - ProcessURL: Dispatches user entered URL to appropriate application
  - StockGraph: Uses custom URL to graph stock data.

Kyocera APIs

- **Kyocera libraries**
  - The *pdQCore* and *pdQRegistry* libraries are automatically loaded on device startup
  - Calling a Kyocera library function:
    1. Include *pdQCore.h* or *pdQRegistry.h*
    2. Call *SysLibFind* to obtain a reference number to the library

```
<snip from pdQCore.h>
#define PDQCoreLibName                  "pdQCore Library"
<snip>

static void AppInit(void)
{
   UInt16 coreRefNum;
   Err err = 0;
   // request a reference number to a Kyocera library
   err = SysLibFind(PDQCoreLibName, &coreRefNum);
   if (!err)
      // proceed to call Kyocera-specific APIs
}
```

  - **pdQ Signal API**: Register for notification of signals such as call states, operating states, messaging
  - **pdQ Phone Module API**: Access phone functionality such as power, earpiece volume, phone#
  - **pdQ Telephony API:** Telephony functionality such as originating, ending or answering a call
  - **pdQ Registry API**: URL and macro (URL bundled with a name and description) handling

Palm

http://www.palmos.com/dev/





- **Expansion and Universal Connector**
  - m500, m505, m125: Palm OS 4.0
  - Features Include: SC/MMC expansion slot, Universal Connector
- **Wireless**
  - VIIx: Palm OS 3.5
  - Features Include: web clipping, wireless email
- **Additional Handhelds**
  - m105 (OS 3.5), m100 (OS 3.5), Vx (OS 3.5), IIIx (Color, OS 3.5)
- **Development Tools include**
  - PluggedIn Program: targets developers wishing to utilize expansion technologies
  - Palm OS SDK 4.0: headers, libraries, samples for Palm OS development
  - Palm OS Emulator
  - Debug and release ROMs available via Resource Pavilion web page
  - Palm OS 4.0 SDK is included in CodeWarrior for Palm OS v 8.0
- **Sample Code includes (SDK 4.0 samples)**
  - Address, Datebook, Memo, ToDo
  - Beamer: implements beaming support
  - Games: MineHunt, Reptoids, SubHunt

Palm APIs

- **SD/MMC**
  - Verify we are running on OS 4.0 or higher (our application should already know the OS version)

```
err = FtrGet(sysFtrCreator,sysFtrNumROMVersion, &romVersion);
//The romVersion parameter should be greater than or equal to 0x04003000
sysMakeROMVersion(4,0,0,sysROMStageRelease,0)
```

  - Check for Expansion Manager and VFS Manager (VFS implies existence of Expansion)

```
err = FtrGet(sysFileCExpansionMgr, expFtrIDVersion, &expMgrVersion);
if(err)
  // Expansion Manager is not installed
else
  // check the version of Expansion Manager(expMgrVersion) if necessary

// VFS Manager FtrGet:
// err = FtrGet(sysFileCVFSMgr, vfsFtrIDVersion, &vfsMgrVersion);
```

  - We are now able to make Expansion and VFS Manager calls and poll any available slots
- **Universal Connector**
  - Detecting peripheral attachments with notifications (register with SysNotifyRegister)
    - sysExternalConnectorAttachEvent:
      identifies type of UC attached device, only broadcast on devices w/Universal C
    - sysExternalConnectorDetachEvent:
      identifies type of UC detached device, only broadcast on devices w/Universal C

Sony

http://www.us.sonypdadev.com/top.html

- **PEG-N Series**
  - PEG-N760C (OS 4.1), PEG-N710C (OS 3.5.2), PEG-N610C (OS 4.0)
  - Features Include: 320x320 resolution, 16 bit color, Jog Dial, Memory Stick
- **PEG-T Series**
  - PEG-T615C (OS 4.1), PEG-T415 (OS 4.1)
  - Features Include: 320x320 resolution, Jog Dial, enhanced IR, Memory Stick
- **PEG-S Series**
  - PEG-S320 (OS 4.0), PEG-S360 (OS 4.0)
  - Features Include: Jog Dial, Memory Stick
- **Development Tools include**
  - Sony SDK 3.0/2.0: header files, Programming Companion documentation
  - POSE 3.0a8 (PEG-S and PEG-N), POSE 3.2 (PEG-T)
  - ROMs available via Sony Developer web page
  - HDKs: contain connection diagrams and mechanical information
  - Sony CLIE SDK 3.0e is included in CodeWarrior for Palm OS v 8.0
- **Sample Code includes (Sony SDK)**
  - Jog Assist Sample: support of Jog Assist functionality





Sony APIs

- **Jog Dial™ Navigator**
  - Ensuring the Jog Dial Navigator is available

```
#include <SonyCLIE.h>
...
if(!FtrGet(sonySysFtrCreator, sonySysFtrNumSysInfoP, (UInt32 *)&infoP)) {
  if (infoP && (infoP->extn & sonySysFtrSysInfoExtnJog)) {
    /* Jog Dial function is available */
  } else { /* Jog Dial is NOT available */ }
```

- **High Resolution (320x320)**
  - Ensuring High Resolution is available

```
// assume FtrGet returned no error, therefore a Sony device
if (sonySysFtrSysInfoP->libr & sonySysFtrSysInfoLibrHR) {
   /* HR available */
   if ((error = SysLibFind(sonySysLibNameHR, &refNum))){
      if (error == sysErrLibNotFound) {
         /* couldn't find lib */
         error = SysLibLoad( 'libr', sonySysFileCHRLib, &refNum );
      }
   }
   if (!error ) /* Now we can use the High Resolution library */
```

Sony APIs

- **Memory Stick ®**
  - Sony documentation recommends usage of FtrGet for determining the presence of a Memory Stick

```
#include <SonyCLIE.h>
...
if(!FtrGet(sonySysFtrCreator, sonySysFtrNumSysInfoP, (UInt32 *)&infoP)) {
  if (infoP) {
    if (infoP->msStatus & sonySysFtrSysInfoMsStatus1StrgMS)
       // Physically formatted Memory Stick media is inserted in slot 1
    if (infoP->msStatus & sonySysFtrSysInfoMsStatus1MGMS)
       // MagicGate™ Memory Stick media is inserted in slot 1
    ...
```

  - Additionally, register for Memory Stick inserted notifications if appropriate
  - Using the Notification Manager API function SysNotifyRegister, we can register for
    - sysNotifyCardInsertedEvent: Memory Stick is inserted into the CLIE
    - sysNotifyCardRemovedEvent: Memory Stick is removed from the CLIE
    - sysNotifyVolumeMountedEvent: Memory Stick file system is mounted correctly
    - sysNotifyVolumeUnmountedEvent: Memory Stick file system is unmounted

Symbol

http://devzone.symbol.com



- **Devices**
  - STP 1500: Palm OS 3.0, bar code scan engine
  - STP 1550: miniature scan engine, scan triggers
  - STP 1700: Palm OS 3.5, wireless LAN, SE 900/SE 900HS scan engine
  - STP 1733/1734: Palm OS 3.5, wireless WAN, SE 900 scan engine
  - STP 1800: wireless WAN, wireless LAN, miniature scan engine

- **Development Tools include**
  - Symbol SDK v3.10: documentation, headers, libraries
  - ROMs available via Symbol developer zone
  - SPT ROM Builder Utility v3.0x: development of custom ROMs
  - Documentation: Terminal Scanner System, Printer, Driver extensions, etc.
  - Symbol Terminal SDK Version 3.10 is included in CodeWarrior for Palm OS v 8.0



- **Sample Code includes (Symbol SDK v3.10)**
  - ScanDemo and Sscan: develop scan-aware 1D applications
  - Print Sample: usage of the printer API
  - MSR Sample: usage of the Magnetic Strip Reader ability
  - S24 Samples: examples of Spectrum24 code

Symbol APIs

- **Using the Scan Manager Shared Library**
  - **Include *ScanMgr.lib* in your project**
  - **Call *ScanIsPalmSymbolUnit* to determine if we are on a Symbol device**
  - **Call *ScanOpenDecoder* to load and initialize the Scan Manager shared library**
  - **Set global variable to indicate success**

```
static void StartApplication(void)
{
	Err error;

	// first check and be sure we're running on Palm/Symbol hardware
	if (ScanIsPalmSymbolUnit())
	{
		error = ScanOpenDecoder();	// open the scan manager library
		if (!error)
		{
			bOpenDecoderOK = true;	// set global variable that all is well
			ScanCmdScanEnable();	// Enable scanning
		}
		else
			FrmAlert( OpenFailedAlert );
	}
	else
		FrmAlert( WrongHardwareAlert );	// Note: is an alert appropriate?

	FrmGotoForm( MainForm );
}
```

Extensible Coding for varied screen density

Assume: Application currently supports 160x160 monochrome screens

Assume: We have verified existence of the feature with the Feature Manager, etc.

Palm has added a double-density API on OS 5 to be used on future devices

- **QVGA and Virtual Graffiti: HandEra SDK**
  - **Palm OS applications should run without modification**
  1. Create high resolution icons, bitmaps,
  2. Create high resolution forms at compile-time or run-time via API calls
  3. Add a QVGA ‘sKst’ resource
  4. Add display extent event-handling for resizing, etc.

- **High Resolution (320x320): Sony SDK**
  - **Palm OS applications should run without modification**
  1. Call `HRWinScreenMode()` to initiate high-resolution mode
  2. Call appropriate High resolution API functions
  3. Exit high-resolution mode on application exit or switch

Extensible Coding: unique input technology

Assume: Application currently supports standard user input features

- **Auxiliary Button and Jog Wheel: HandEra SDK**
  - **Palm OS applications should run without modification**
  1. Include *TrgChars.h* in the project
  2. Add code to handle the following keyDownEvents:
     vchrPrevField,vchrNextField, chrCarriageReturn, chrEscape

- **Jog Dial Navigator: Sony SDK**
  - **Palm OS applications should run without modification**
  - Optionally, check preferences for `sonySysFtrSysInfoExtnJogAst`
    Jog Assist will process unmasked jog events as documented in the SDK
  1. Include *SonyChars.h* in the project
  2. Add code to handle the following keyDownEvents:
     vchr+ JogUp, JogDown, JogPush, JogPushRepeat, JogRelease, JogPushedUp, JogPushedDown, JogBack

- **NOTE: There is an overlap in the virtual character values of the key events generated by the Sony Jog Dial hardware and the clock silk-screened button of the Palm m500/m505**

  Palm Developer Knowledge Base Article (http://oasis.palm.com/dev/kb/faq/2709.cfm)

Device Detection: a tricky task

- Caveats
  - Your application should be polling features
  - Never assume features based on device model
  - Detection of specific devices is dangerous
  - Cross-check against features
- All devices running Palm OS® Version 3.5 or later can utilize FtrGet
  - `sysFtrNumOEMCompanyID:` `('trgp','palm','sony',etc.)`
  - `sysFtrNumOEMDeviceID:` `('trg2','trnd','ecty',etc.)`
  - `sysFtrNumOEMHALID:` `('sonyHwrOEMHALID_S300',etc.)`
  - Examine *HwrMiscFlags.h* in the Palm SDK for further details
- A device detection scheme pre Palm OS 3.5 devices is dependent on the device (http://mobilegeographics.com/dev/devices.html)
- Summary: Check features and cross-check against features if device detection is necessary



Session 201: Targeting All Palm Powered Devices: QA



PalmSource®

Conference and Expo